

# Understanding Different Types of APIs

A beginner-friendly technical guide covering five common API styles and how they differ.

## 1. REST APIs

**REST (Representational State Transfer)** is one of the most widely used approaches for building web APIs. REST APIs use standard HTTP methods to operate on resources, which are commonly represented as JSON objects.

### Common HTTP methods

- GET: Retrieve a resource or collection of resources.
- POST: Create a new resource or submit data for processing.
- PUT: Replace or update an existing resource.
- PATCH: Partially update an existing resource.
- DELETE: Remove a resource.

**Example:** A student-management API might expose GET /students/42 to retrieve the student with ID 42.

REST is popular because it works naturally with HTTP, is relatively simple to understand, and is supported by almost every modern programming language and web framework.

## 2. SOAP APIs

**SOAP (Simple Object Access Protocol)** is a protocol for exchanging structured messages between applications. SOAP commonly uses XML for requests and responses and can operate over HTTP and other transport protocols.

SOAP has strict standards for message structure, security, transactions, and error handling. This makes it useful in environments where formal contracts and enterprise-level reliability are important.

### Typical characteristics

- XML-based message format.
- Strongly defined service contracts, often described using WSDL.
- Built-in standards for features such as security and transactions.
- More verbose than many modern REST APIs.

**Example:** A banking system may expose a SOAP service for processing a financial transaction where strict schemas and security requirements are important.

## 3. GraphQL APIs

**GraphQL** is a query language and API runtime that allows clients to request exactly the data they need. Instead of relying on many fixed REST endpoints, a GraphQL API commonly exposes a single endpoint through which clients send structured queries.

### Example query

```
{
  student(id: 42) {
    name
    course
    email
  }
}
```

The client controls which fields it wants returned. This can reduce over-fetching, where an API returns more information than the application needs. GraphQL is particularly useful for applications with complex or changing data requirements.

### Important concepts

- Queries retrieve data.
- Mutations modify data.
- Schemas describe available types and operations.
- Resolvers determine how requested data is obtained.

## 4. gRPC APIs

**gRPC** is a high-performance remote procedure call framework originally developed at Google. It commonly uses HTTP/2 for transport and Protocol Buffers (protobuf) to define services and serialize messages.

Instead of thinking primarily in terms of web resources, developers define services and methods. A client can then call a remote method as if it were a local function.

### Example service definition

```
service StudentService {
  rpc GetStudent(GetStudentRequest) returns (Student);
}
```

gRPC is commonly used for communication between internal services in distributed systems. Its efficient binary serialization and HTTP/2 features can provide strong performance, although it is generally less convenient than REST for direct browser-based API consumption.

## 5. WebSocket APIs

**WebSocket** APIs provide a persistent, two-way communication channel between a client and server. After a connection is established, either side can send messages without waiting for a new HTTP request each time.

This makes WebSockets useful when applications need near real-time updates.

## **Common use cases**

- Chat applications.
- Live notifications.
- Multiplayer games.
- Real-time dashboards.
- Live collaboration tools.

**Example:** In a chat application, a server can push a newly received message to connected clients immediately instead of requiring each client to repeatedly request new messages.

# Comparing the Five API Types

| API type  | Typical format / protocol       | Best suited for                             |
|-----------|---------------------------------|---------------------------------------------|
| REST      | HTTP + JSON                     | Web and mobile applications, public APIs    |
| SOAP      | XML + SOAP protocol             | Enterprise systems, formal contracts        |
| GraphQL   | GraphQL queries                 | Flexible and data-heavy client applications |
| gRPC      | HTTP/2 + Protocol Buffers       | Fast service-to-service communication       |
| WebSocket | Persistent WebSocket connection | Real-time, two-way communication            |

## How to Choose an API Style

There is no universally best API type. The appropriate choice depends on the application's requirements, clients, performance needs, and development environment.

- Choose **REST** when you want a straightforward HTTP API that is easy for many clients to consume.
- Choose **SOAP** when strict contracts and established enterprise standards are important.
- Choose **GraphQL** when clients need flexible control over complex data queries.
- Choose **gRPC** when efficient service-to-service communication and strong contracts are priorities.
- Choose **WebSocket** when the server and client need continuous, real-time, two-way communication.

## Key Takeaway

REST, SOAP, GraphQL, gRPC, and WebSocket APIs solve different communication problems. REST is resource-oriented and HTTP-friendly, SOAP emphasizes formal messaging standards, GraphQL gives clients control over requested data, gRPC focuses on efficient remote procedure calls, and WebSockets enable persistent real-time communication.

**Document purpose:** This document is original sample content created for testing document retrieval, chunking, embeddings, and question answering in a RAG application.